Case Study on Analysis of Data Communication in a Web Application using Browser Developer Tools

Name:	Irwin Inbadurai V.J	**Reg. No.:**	241201018
Course:		**Date:**	

Aim

To analyse how a browser and a web server exchange data, by inspecting requests, responses, headers, cookies and timings of a web application with the browser's developer tools.

Objectives

- Open and use the Network, Application and Console panels of the browser developer tools.
- Identify the parts of an HTTP request and response.
- Interpret status codes and the timing breakdown of a request.
- Examine cookies and security-related headers and relate observations to the network layers.

Selected Web Application

Online registration form (illustrative). A web application shows a login page, accepts a username and password, and then loads a dashboard that fetches data in the background. Such an application could be built on any platform, including a Zoho Creator form application as studied in the earlier experiments. The aim is to see what actually travels between the browser and the server when a user clicks a button.

Developer Tools Panels Used

Panel	How to open	Use in this study
Elements	F12 or Ctrl+Shift+I	Inspect the page structure and form fields
Console	Same shortcut, Console tab	See script errors and logged messages
Network	Same shortcut, Network tab	Record every request and response
Application	Same shortcut, Application tab	View cookies, local storage, session storage
Security	Security or lock icon	Check HTTPS and certificate details

Procedure

Step	Action
1	Open the application in a fresh browser tab, then open the developer tools and select the Network tab.
2	Tick **Preserve log** and **Disable cache**, and clear any earlier records.
3	Reload the login page and note the list of requests.
4	Enter test credentials and submit the form; locate the request that carries the form data.
5	Click the request and study the Headers, Payload, Response and Timing tabs.
6	Open the Application tab to see the cookies set after login.
7	Throttle the connection to a slow profile and reload to see its effect on load time.

Anatomy of an HTTP Exchange

Part	Request (browser to server)	Response (server to browser)
First line	Method, path, protocol version (for example POST /login)	Protocol version and status code
Headers	Host, User-Agent, Accept, Content-Type, Cookie	Content-Type, Content-Length, Set-Cookie, Cache-Control
Body	Form data or JSON (for POST and PUT)	HTML page, JSON data, image or file

Requests Recorded (assumed sample)

Name	Method	Status	Type	Size	Time
login (page)	GET	200	document	6.2 kB	180 ms
styles.css	GET	200	stylesheet	14 kB	60 ms
app.js	GET	200	script	95 kB	140 ms
logo.png	GET	304	png	cached	25 ms
login (submit)	POST	302	document	0.4 kB	210 ms
dashboard	GET	200	document	11 kB	190 ms
api/records	GET	200	fetch / JSON	3.8 kB	120 ms
missing-icon.svg	GET	404	svg	0.3 kB	40 ms

Status Codes Encountered

Code	Meaning	Where seen above
200 OK	Request succeeded	Page, CSS, JavaScript, data
302 Found	Temporary redirect to another URL	After successful login
304 Not Modified	Use the cached copy	Logo image
404 Not Found	Resource does not exist	Missing icon file (a defect to fix)

Timing Breakdown of One Request

Phase	Meaning	Example (assumed)
Queueing / Stalled	Waiting before the request can start	5 ms
DNS lookup	Convert host name to IP address	20 ms
Initial connection	TCP three-way handshake	30 ms
SSL / TLS	Negotiate encryption	40 ms
Request sent	Upload request to server	1 ms
Waiting (TTFB)	Server processing until first byte arrives	90 ms
Content download	Receive the response body	24 ms

A long *Waiting* time points to a slow server or database. A long *Content download* points to a large file or a slow network. A long *DNS lookup* or *Initial connection* points to name resolution or network distance problems.

Cookie and Header Observations

Item	Observation	Why it matters
Protocol	Site is served over HTTPS	Data is encrypted in transit
Set-Cookie flags	Session cookie marked Secure, HttpOnly, SameSite	Reduces theft by scripts and cross-site misuse
Cache-Control	Static files cached, API responses not cached	Speed for static content, fresh data for records
Content-Type	API returns JSON	Client script can parse data directly
Form data	Password visible in the Payload tab of the user's own browser	Normal; it is encrypted on the wire by HTTPS

Relation to the Network Layers

Observation in DevTools	Layer concerned
URL, methods, headers, JSON	Application (HTTP)
TLS negotiation time	Presentation / security (TLS)
Initial connection time, ports 80 and 443	Transport (TCP)
IP address of the server shown in the connection details	Network (IP)
Load time on throttled connection	Physical and link capacity

Findings and Improvements Suggested

- The 404 for the missing icon should be fixed to avoid a wasted request.
- Combining or compressing JavaScript would reduce the largest download.
- Caching headers on static files already reduce repeat load time, as the 304 response shows.
- A server waiting time above about 100 ms for data requests would justify checking the database query.

Results

- Every user action could be traced to one or more HTTP requests with a clear cause and effect.
- Status codes and timings made faults, such as the missing file, easy to locate.
- Cookie and header checks gave a quick first view of the application's security basics.

Conclusion

Browser developer tools act as a built-in network analyser for the application layer. They let a student or developer see the conversation between browser and server without extra software. The experiment confirms that web communication is a chain of request and response pairs, and that performance and security problems can be found by reading status codes, timings, headers and cookies.

Further reading: Cisco Networking Academy material for Packet Tracer; browser vendor documentation for developer tools (Chrome DevTools Network panel); IETF specifications for TCP, IP and HTTP. The office network, the web application and all timing or size values in this report are illustrative and assumed to demonstrate the method; they should be replaced with the values you observe in your own practical.